Envisioner — Corrected Master Handoff

Important correction

The original four modules are the four modules listed below. The previous handoff incorrectly grouped several implementation areas as the four product modules. Those areas are supporting subsystems inside the modules, not the original product architecture.

Do not describe the supporting subsystems as the original four modules.

Product vision

Envisioner is intended to become an industry-ready architectural/home-design platform with four major product modules:

1. Manual 3D Home Builder.
2. 2D Floor Plan to 3D Auto-Conversion.
3. AI-Generated House Designs.
4. AR/VR House Tour.

Module 1 is built first because it provides the 3D rendering engine and canonical structured home data required by Modules 2, 3, and 4.

Module 1 — Manual 3D Home Builder

Purpose

This is the foundation and first build target. It provides a demoable home configurator and the canonical data model for walls, doors, windows, rooms, floors, stairs, roofs, materials, and objects.

Technology direction

- Use Three.js or React Three Fiber for browser 3D rendering.
- Keep the house represented as structured JSON/model data, not only raw geometry.
- Reuse the same schema in Modules 2 and 3.
- Use the 2D canvas as a CAD-like editing surface.
- Use real-time 3D rendering from the same model.

Core data model direction

The model should support at least:

```
{
  schemaVersion,
  rooms: [],
  floors: [],
  walls: [
    {
      id,
      floorId,
      start: { x, y },
      end: { x, y },
      height,
      thickness,
      material
    }
  ],
  openings: [
    {
      id,
      floorId,
      wallId,
      type: "door" | "window",
      offsetFromWallStart,
      width,
      height,
      sillHeight,
      asset/material
    }
  ],
  stairs: [],
  roof: {
    type: "flat" | "gable" | "hip",
    settings: {}
  },
  furniture: [],
  materials: {},
  settings: {}
}
```

The current implementation uses floor-level collections for several entity types. Preserve compatibility while gradually moving toward a canonical project schema.

Manual builder capabilities

2D editing

- Draw walls on a grid.
- Snap to grid and useful angles.
- Select walls.
- Drag wall endpoints to lengthen or shorten walls.
- Move walls.
- Place doors/windows at positions along walls.
- Store opening position as offset from wall start.
- Move doors/windows along host walls.
- Place, move, rotate, resize, and delete furniture.
- Place, move, rotate, and delete stairs.
- Detect closed room loops.
- Validate room boundaries and object placement.

3D generation

- Extrude wall lines into 3D boxes using Three.js geometry.
- Support door/window openings in walls.
- Use CSG such as `three-csg-ts` where appropriate for true wall holes.
- Render floors, slabs, ceilings, roofs, furniture, doors, windows, and stairs.
- Keep 2D and 3D geometry synchronized.

Roofs

Initial roof presets:

- Flat.
- Gable.
- Hip.

Use parametric generators instead of free-form modeling in the initial product. Roof and ceiling planes must remain geometrically correct and level unless a deliberate roof type requires otherwise.

Materials

- Use Three.js `MeshStandardMaterial`.
- Support color changes and swappable textures.
- Initial materials can include paint, brick, wood, tile, and similar presets.
- Keep material handling simple and data-driven instead of building a full material-authoring engine initially.

Stairs completed in Module 1

Supported types:

- Straight.
- L-shaped.
- U-shaped.

Capabilities:

- Connect active floor to the immediately higher real floor.
- Block placement on the topmost floor.
- Store explicit stair schema.
- Support width, run, landing length, turn direction, step count, color/material, position, and rotation.
- Use accurate multi-part footprints.
- Validate against rooms, walls, doors, windows, furniture, and other stairs.
- Show ghost previews and persistent red warnings.
- Show a destination-floor arrival guide.
- Create a procedural destination slab/ceiling opening shaped to the true stair footprint.
- Render real individual 3D steps and landings.
- Support selection, movement, free rotation, R-key rotation, deletion, and undo/redo.
- Tag 3D groups for future GLB export.

Module 2 — 2D Floor Plan to 3D Auto-Conversion

Purpose

Convert an uploaded 2D floor-plan image into the same structured data used by Module 1, then render it through the existing 3D pipeline.

This is not a separate rendering engine. It should auto-fill Module 1's data model.

Standard pipeline

1. Image classification and cleanup.
    - Confirm the upload resembles a floor plan.
    - Normalize scale, orientation, resolution, and contrast.
    - Identify whether a scale reference is present.
2. Structural element detection.
    - Detect walls.
    - Detect doors.

    - Detect windows.
    - Detect rooms or room boundaries.
    - Candidate technologies include CNN/object-detection models such as YOLO-family models or RF-DETR, depending on practicality and licensing.
3. Vectorization.
    - Convert detected pixels into clean lines and polygons.
    - Infer wall centerlines and thickness.
    - Detect opening positions relative to walls.
    - Clean noise, gaps, duplicate segments, and incorrect joins.
4. 3D reconstruction.
    - Convert vectorized walls into `walls[]` entries.
    - Convert detected doors/windows into `openings[]` entries.
    - Infer rooms and floor-plan loops.
    - Extrude using Module 1's renderer.
    - Insert existing door/window assets.
    - Scale to real dimensions using labels or a user-provided reference.

Recommended starting point

Do not train a model from scratch initially.

- Study and use the CubiCasa5k dataset and associated open-source pipelines as a baseline.
- Evaluate pretrained wall/room detection models.
- Start with a limited supported input style.
- Add manual correction tools after automatic detection.
- Always let the user review and correct vectorized walls/openings before final 3D conversion.

Module 2 product flow

1. User uploads a floor-plan image.
2. System validates and preprocesses it.
3. System detects candidate walls, doors, windows, and rooms.
4. System displays a 2D review/correction layer.
5. User adjusts scale and corrects geometry.
6. System converts accepted results into the Module 1 schema.
7. Existing Module 1 3D rendering displays the result.
8. User continues editing manually.

Module 2 quality requirements

- Never present uncertain detections as guaranteed facts.
- Show confidence or review status where practical.
- Preserve source-image coordinates and generated-model coordinates.
- Allow re-running detection without destroying manual edits.
- Keep imported geometry tagged as imported/generated for traceability.
- Handle scale uncertainty explicitly.

Module 3 — AI-Generated House Designs

Purpose

Generate multiple possible house/floor-plan layouts from dimensions, room requirements, orientation, and user preferences, then allow the user to select and convert a preferred 2D option into the Module 1/2 structured model and 3D view.

The initial target is 2D layout generation, not direct photorealistic 3D generation.

Preferred architecture

Use constraint-conditioned generation rather than unconstrained image generation.

Candidate approaches:

Approach	Role	Suitability
Stable Diffusion + ControlNet	Generate visual variations constrained by a boundary/sketch	Useful for quick style/layout variations
Fine-tuned Stable Diffusion + LoRA	Adapt generation to floor-plan styles and structured prompts	Useful after collecting/curating data
ChatHouseDiffusion-style pipeline	LLM parses requirements into a room graph, then a diffusion system generates layouts respecting it	Closest match to dimension/preference-driven layouts

Recommended product flow

1. User enters plot/building dimensions.
2. User enters room requirements, for example bedrooms, bathrooms, kitchen, living room, parking, stairs, and preferences.
3. User specifies constraints such as orientation, entrances, circulation, approximate room sizes, and required adjacency.
4. An LLM converts the request into a structured requirement graph.
5. A layout generator creates several 2D candidate plans.
6. A constraint validator filters or flags candidates.

7. User compares options.
8. User chooses one option.
9. The chosen option is converted into Module 1's structured data model.
10. Existing 2D/3D editing continues from that result.

Module 3 constraints

The generator should account for:

- Outer boundary and plot dimensions.
- Room count and minimum areas.
- Room adjacency.
- Circulation paths.
- Door placement.
- Window placement where possible.
- Stair placement and floor connections.
- North-facing/orientation requirements.
- User preferences.
- Feasibility warnings.

AI output must be editable structured geometry, not only a pretty raster image.

Module 3 implementation strategy

- Start with rule-based or graph-based layout generation if needed for reliable MVP behavior.
- Add AI generation once the structured model and validation rules are stable.
- Generate 2D first.
- Reuse Module 1's geometry and rendering pipeline for 3D.
- Do not attempt direct AI-generated 3D on the first version.

Module 4 — AR/VR House Tour

Purpose

Allow users to inspect and tour the generated house in AR, VR, and first-person 3D modes.

AR direction

- Export Module 1/2/3 models as glTF/GLB.
- Use the web-standard glTF/GLB format.
- Embed `<model-viewer>` with AR enabled for supported devices.

- Provide placement of the model into the user's physical environment where browser/device support allows it.

Walkthrough direction

- Add first-person camera controls to the existing Three.js scene.
- Layer movement and collision logic over the same rendered building model.
- Let users walk through rooms rather than only orbit around the model.
- Add collision boundaries so the camera cannot pass through walls.
- Include floor switching/elevator or stair movement behavior if desired.

VR direction

- Treat true headset support as a later stretch goal.
- Use WebXR with Three.js or compatible model-viewer support.
- Prioritize AR and desktop/mobile first-person walkthrough before full VR.

Module 4 product flow

1. User completes or imports a house model.
2. User previews it in desktop 3D.
3. User exports or opens an AR/VR-ready version.
4. Supported device opens AR placement or a first-person tour.
5. User navigates the building and views materials, rooms, stairs, roofs, and furniture.

3. Supporting subsystems across all modules

These are not the original four product modules. They support the modules above.

Canonical data model

- Versioned project schema.
- Floors, rooms, walls, openings, stairs, roofs, furniture, materials, settings.
- Stable IDs.
- Source/provenance metadata for manually created, imported, or AI-generated geometry.

Geometry engine

- 2D polygons, wall segments, room loops, footprints, intersections, snapping, tolerances.
- Accurate 2D/3D coordinate conversion.
- Shared geometry functions for validation and rendering.

Validation engine

- Pure shared validation for Canvas2D and View3D.
- Floor-aware checks.
- Collision reasons and warning sets.
- Boundary-aware point-in-polygon.
- Symmetric object collision warnings.
- Preservation of invalid objects with visible warnings.

Interaction system

- Selection by entity type.
- Drag/move/rotate/resize.
- Pointer capture cleanup.
- Ghost previews.
- Escape/mode/floor/view cancellation.
- One history entry per completed gesture.

Persistence

- Save/load.
- Schema migration.
- Local storage/project files first.
- Future cloud storage.
- Autosave/recovery later.

Export

- GLB generation.
- Exclude editor-only guides, handles, overlays, and previews.
- Preserve materials and object identity.
- Include correct stair openings and geometry.

Testing and quality

- Unit geometry tests.
- Validation tests.
- Browser interaction tests.
- Manual visual regression screenshots.
- Console/runtime error checks.

- Cross-browser checks.
- Large-project performance checks.

4. Completed work in Module 1

Completed and verified:

- Multi-floor manual home builder.
- 2D wall editing.
- Doors and windows.
- Furniture placement and editing.
- Roof/ceiling/cantilever behavior.
- 2D and 3D views.
- Normal, X-Ray, Hide Walls, and All Floors modes.
- Centralized validation.
- Multi-type stairs.
- Stair ceiling openings.
- Wall endpoint editing.
- Furniture wall clamping.
- Rotation consistency between 2D and 3D.
- Immediate collision warnings.
- Undo/redo gesture behavior.
- Blue-screen render-crash fix.

5. Future roadmap

Near-term

1. Lock down reliable browser/manual regression testing.
2. Add geometry and validation unit tests.
3. Add project schema versioning.
4. Implement Save/Load.
5. Verify round-trip preservation of every entity, especially stairs and floor links.

Medium-term

6. Implement robust GLB export.
7. Add imported floor-plan workflow for Module 2.
8. Build image preprocessing and vectorization review tools.
9. Add manual correction after AI detection.

Long-term

10. Add Module 3 structured AI layout generation.
11. Add candidate comparison and constraint scoring.
12. Add Module 4 AR using model-viewer.
13. Add first-person walkthrough mode.
14. Add WebXR VR as a stretch goal.

6. Rules for future implementation

- Module 1 remains the foundation for Modules 2, 3, and 4.
- New modules must feed or consume the canonical structured model.
- Do not create a separate incompatible 3D renderer for Modules 2 or 3.
- Do not generate only images when the result must be editable geometry.
- Preserve completed stair behavior unless a reproducible regression is demonstrated.
- Verify shared-code changes across walls, doors, windows, furniture, stairs, roofs, and floors.
- Do not claim browser behavior is fixed from code reading alone.
- Report file changes, root causes, and actual test method.
- Do not push, commit, deploy, or upload to GitHub without explicit permission.

7. New-thread opening prompt

> Continue development of Envisioner using the attached `envisioner-master-handoff-corrected.md`. This is the authoritative product context. The original four product modules are exactly: Module 1 Manual 3D Home Builder, Module 2 2D Floor Plan to 3D Auto-Conversion, Module 3 AI-Generated House Designs, and Module 4 AR/VR House Tour. Do not redefine these modules as implementation subsystems.
> Module 1, including the multi-type stair system and documented bug fixes, is complete and verified. Read the handoff and inspect the current repository before asking basic context questions. Do not rewrite completed behavior unless a reproducible regression is shown.
> The next requested task is: [describe task]. Before coding, provide a focused plan with exact files, data flow, validation impact, undo/redo impact, and verification steps. Do not push, commit, deploy, or upload to GitHub.